Day 106: Decision-Tree Pruning — Real Over- and Underfitting

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 2 | Complete study guide

Watch: How to Prune Regression Trees, Clearly Explained!!! (StatQuest) | Cost Complexity Pruning (Theory + Code) (Meerkat Statistics)

Script: `day106_pruning.py` · every number in this guide is copied by program from `day106_run_output.txt`, the real output of that script.

Cost-complexity pruning written from scratch on Day 105's tree. The pruning path reaches the lowest possible cost in 1000 of 1000 checks against every possible pruned tree. On Titanic, walking from one leaf to the full 145 leaves shows both failures: training accuracy keeps rising to 0.9823 while test accuracy rises to 0.8172 and falls back to 0.7537. Over 30 splits pruning adds +0.0397 on Titanic and +0.0076 on the cleaner breast-cancer data. With 30% of the labels flipped, the gain reaches +0.2392.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

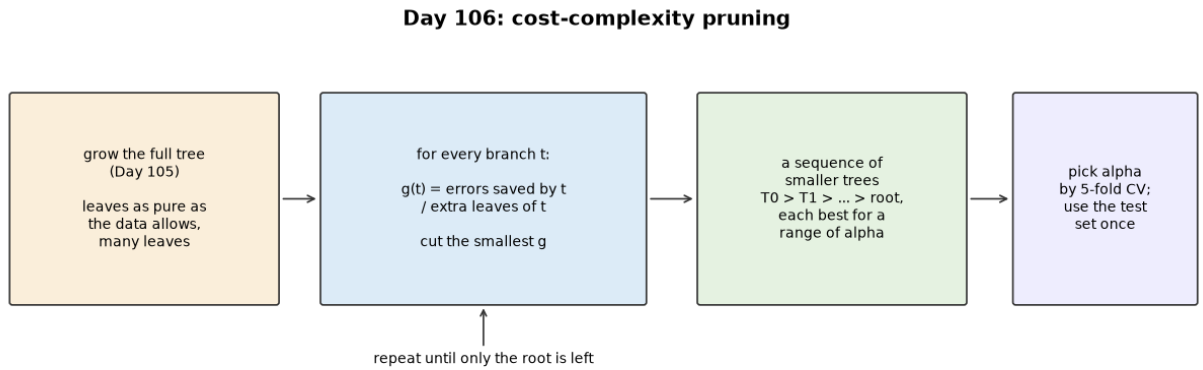
Underfitting: the model is too simple to capture the pattern, so it does badly even on its training data. **Overfitting**: it has memorised details of the training data, so it does well there and worse on new data.

Pruning grows the whole tree first, then cuts back the branches that were not worth their size, and so moves a tree from overfitting towards underfitting one cut at a time. Day 105's depth limit stops growth in advance; pruning decides afterwards.

REAL-WORLD SCENARIO

An editor lets a writer finish the whole draft, then removes the paragraphs that add least for their length. Deciding before writing, as a depth limit does, would lose paragraphs that only look weak at the start.

SYSTEM DIAGRAM



Cost-complexity pruning: grow, cut the weakest link repeatedly, choose along the path by cross-validation.

Today's part for Day 111

`day106_exercise.py` is the weakest-link path in a few short functions, checked on a three-leaf tree whose alphas can be worked out on paper. Its real output:

ACTUAL OUTPUT

```
hand check: leaves and errors along the path
  from alpha 0.000: 3 leaves, 2 errors
  from alpha 0.100: 2 leaves, 4 errors
  from alpha 0.200: 1 leaves, 8 errors
left branch saves 2 errors of 20 with 1 extra leaf: 2 / 20 / 1 = 0.100; root then saves 8 - 4 = 4:
    4 / 20 / 1 = 0.200
at alpha 0.05 -> 3 leaves; at 0.1 -> 2; at 0.5 -> 1
the original still has 3 leaves
a single leaf: alphas [0.0], 1 tree
a split that saves no errors (both children keep the majority): cut at alpha 0.000, 1 leaf
```

Data: Titanic (the repository's own Week 3 copy) and the Breast Cancer Wisconsin set bundled with scikit-learn. Only its data loader is used, no scikit-learn models. Section 7 uses synthetic data on purpose, and says so.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The cost, on a tree you can check by hand

DEFINITION — start here, no prior knowledge assumed

Cost-complexity prices a tree as its training error rate plus a charge α for every leaf. At $\alpha = 0$ leaves are free and the full tree wins; as α rises, leaves stop paying for themselves and the best tree shrinks.

The **weakest link** is the branch that saves the fewest errors per extra leaf. Its g(t) is the α at which keeping it and cutting it cost the same. Cut it, recompute, repeat, and you get a short list of nested trees.

REAL-WORLD SCENARIO

A shop charging rent per shelf: at low rent it stocks everything; as rent rises, the shelf that earns least per square metre goes first.

MATHEMATICS

$$R_\alpha(T) = R(T) + \alpha\,|leaves(T)|, \quad R(T) = \text{training errors of } T / N$$

$$g(t) = (R(t) - R(T_t)) / (|leaves(T_t)| - 1)$$

t = a node as a leaf; T_t = the branch below it; cut the t with the smallest $g(t)$

IMPLEMENTATION

```
def cost_path(tree):
    n, t = int(tree["counts"].sum()), clone(tree)
    alphas, trees = [0.0], [clone(t)]
    while "feature" in t:
        nodes = questions(t)
        g = []
        for q in nodes:
            e, leaves = branch(q)
            g.append((node_errors(q) - e) / n / (leaves - 1))
        a = min(g)
        for q, v in zip(nodes, g):
            if v <= a + 1e-12:
                cut(q)
        alphas.append(a); trees.append(clone(t))
    return alphas, trees
```

ACTUAL OUTPUT

```
  if x0 <= 5.5:
      if x0 <= 3.5:
          -> class 0  [3, 0]
      else:
          if x0 <= 4.5:
              -> class 1  [0, 1]
          else:
              -> class 0  [1, 0]
  else:
      -> class 1  [0, 5]
full tree: 4 leaves, 0 training errors of 10
  from alpha = 0.0000: 4 leaves, 0 errors, error rate 0.00
  from alpha = 0.0500: 2 leaves, 1 errors, error rate 0.10
  from alpha = 0.3000: 1 leaves, 4 errors, error rate 0.40
Actual output from this lesson's run.
```

What it means

Check the first step by hand. The left branch has one error as a leaf and none below it, with 3 leaves, so g = (1 – 0) / 10 / 2 = 0.0500. The inner node saves the same error with only one extra leaf, so its g is 0.1000, and it is never the weakest: the tree goes from 4 leaves straight to 2 leaves. No 3-leaf tree is on the path, because at no α is a 3-leaf tree the cheapest. That is why the path has far fewer trees than there are sizes.

WHY CHOOSE THIS?

One number, α, orders every pruned tree from full to root, so only a short list needs testing.

WHY NOT THIS? (TRADEOFFS)

The error count ignores how confident a leaf is, and the path depends entirely on the tree that was grown.

2. A correctness check against every pruned tree

DEFINITION — start here, no prior knowledge assumed

The claim behind the path is strong: for *any* α, the tree on the path has the lowest cost of *all* possible pruned trees, though only a few of them are ever looked at. On small trees all of them can be listed.

REAL-WORLD SCENARIO

Checking a shortcut route against every possible route on a small map before trusting it on a large one.

IMPLEMENTATION

```
def every_pruning(node):
    leaf = {k: v for k, v in node.items() if k not in ("feature", "threshold", "gain", "left",
        "right")}
    if "feature" not in node:
        return [leaf]
    out = [leaf]
    for a in every_pruning(node["left"]):
        for b in every_pruning(node["right"]):
            out.append({**node, "left": a, "right": b})
    return out
```

ACTUAL OUTPUT

```
40 trees from 40 random Titanic passengers each (depth up to 4): between 7 and 78 possible pruned
    trees
at 25 random alphas each, the path's tree has the lowest possible cost in 1000 of 1000 cases
Actual output from this lesson's run.
```

What it means

The path reached the true minimum in 1000 of 1000 cases, on real passengers, against between 7 and 78 candidates per tree. This does not prove the theorem; it would have exposed a wrong formula or a bug in the cutting, and it did not.

WHY CHOOSE THIS?

A check that does not trust the code it tests.

WHY NOT THIS? (TRADEOFFS)

Only small trees can be enumerated: the count multiplies at every question.

3. Over- and underfitting on Titanic

DEFINITION — start here, no prior knowledge assumed

Every tree on the path is scored on the training and test passengers, from the full tree to a single leaf that predicts 'died' for everyone.

REAL-WORLD SCENARIO

A student's revision notes: one line is not enough, but copying the whole textbook is no help on a new question.

SPECIFICATION

Data	891 passengers, 6 features, Day 105's split: 623 training, 268 test.
Tree	Gini, no depth limit, minimum leaf 1.
Note	The best test row is printed for this table only. It is not used to choose anything.

ACTUAL OUTPUT

```
623 training, 268 test passengers; full tree: 145 leaves, depth 17; 19 trees on the pruning path
training errors of the full tree: 11; unavoidable, because passengers identical in all six features had
    different outcomes: 11
    alpha  leaves  train acc  test acc
  0.00000     145     0.9823    0.7537
  0.00040     127     0.9807    0.7612
  0.00054     103     0.9679    0.7612
  0.00064      93     0.9615    0.7649
  0.00080      65     0.9406    0.7836
  0.00120      44     0.9165    0.8022
  0.00161      24     0.8844    0.8172
  0.00206      17     0.8700    0.7985
  0.00257      10     0.8523    0.8097
  0.00482       7     0.8395    0.8022
  0.00562       5     0.8283    0.7948
  0.01124       4     0.8170    0.7948
  0.17657       1     0.6164    0.6157
highest test accuracy on the path (looked at for this table only, not used to choose): 0.8172 at 24 leaves
Actual output from this lesson's run.
```

What it means

Read upwards from the bottom. One leaf underfits (0.6164 train, 0.6157 test). Adding leaves helps both at first; test accuracy peaks at 0.8172 with 24 leaves. After that, training keeps rising to 0.9823 while test falls to 0.7537: those leaves fit individual passengers. Training never reaches 1 because 11 errors are unavoidable: identical passengers with different outcomes. The test curve is jagged, because one test passenger is worth 0.004.

WHY CHOOSE THIS?

The whole trade-off is visible on one table, using one tree.

WHY NOT THIS? (TRADEOFFS)

Reading the best row off the test set would be cheating; section 4 chooses without it.

4. Choosing α by cross-validation

DEFINITION — start here, no prior knowledge assumed

Each of 5 folds grows its own full tree and its own path, and every candidate α is scored on the rows that fold left out. The candidates are **geometric means** of neighbouring alphas, one value inside each tree's range.

The **one-standard-error rule** takes the simplest tree whose score is within one standard error of the best: if the difference could be noise, prefer the smaller tree.

REAL-WORLD SCENARIO

Two cars cost the same within the error of the quote: take the simpler one, which has less to go wrong.

MATHEMATICS

$\beta_k = \sqrt{(\alpha_k \alpha_{k+1})}, \quad SE = std(fold\ scores) / \sqrt{k}$

one-SE: the largest β *with* $CV(\beta) \geq max\ CV - SE\ of\ the\ best$

IMPLEMENTATION

```
def cv_alpha(X, y, k=5, seed=0):
    n_classes = int(y.max()) + 1
    full = grow(X, y, n_classes, gini)
    alphas, trees = cost_path(full)
    b = betas(alphas)
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), k)
    scores = np.zeros((k, len(b)))
    for i in range(k):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(k) if j != i])
        al, ts = cost_path(grow(X[tr], y[tr], n_classes, gini))
        for j, a in enumerate(b):
            scores[i, j] = accuracy(at_alpha(al, ts, a), X[va], y[va])
    return alphas, trees, b, scores.mean(0), scores.std(0) / np.sqrt(k)
```

ACTUAL OUTPUT

```
  best CV mean : alpha 0.00249, CV accuracy 0.8139 (+- 0.0178), 15 leaves, train 0.8652, test 0.7910
  one-SE rule  : alpha 0.01163, CV accuracy 0.8011 (+- 0.0165), 4 leaves, train 0.8170, test 0.7948
one-SE tree: 95% interval for test accuracy [0.7425, 0.8433] (Day 99's bootstrap)
the one-SE tree as rules (counts are [died, survived] among training passengers):
  if is_female <= 0.5:
      -> died  [330, 75]
  else:
      if Pclass <= 2.5:
          -> survived  [7, 110]
      else:
          if Fare <= 23.4:
              -> survived  [29, 51]
          else:
              -> died  [18, 3]
Actual output from this lesson's run.
```

What it means

The best mean gives 15 leaves and the one-SE rule gives 4 leaves, and on this split the smaller tree scored 0.7948 against 0.7910, inside an interval of [0.7425, 0.8433]. Four leaves cost almost nothing here and can be read in a few seconds: men died; women in first or second class survived; women in third class survived unless the fare was high. Section 5 asks whether this holds beyond one split.

WHY CHOOSE THIS?

Uses only the training part, and gives a small tree that can be read.

WHY NOT THIS? (TRADEOFFS)

Five folds on 623 rows is noisy: the standard error, about 0.017, is larger than many gaps it decides.

5. Is it better? Thirty splits

DEFINITION — start here, no prior knowledge assumed

One split cannot compare methods, so the whole procedure runs on 30 random splits. Each method makes its choice by 5-fold cross-validation on that split's training part, and the test part is used once. **Paired** differences compare methods on the same split.

REAL-WORLD SCENARIO

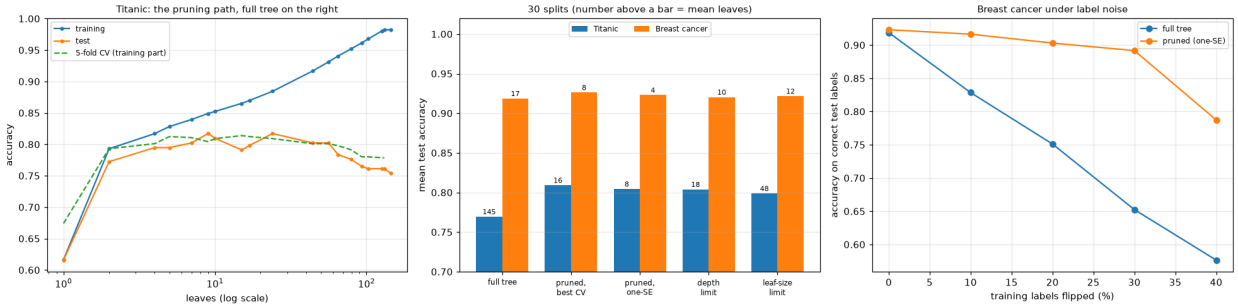
Testing two fertilisers on thirty pairs of neighbouring plots, rather than one field each.

SPECIFICATION

Methods	Full tree; pruned (best CV, and one-SE); depth limit; minimum leaf size.
Grids	Depth 1 to 10 or none; leaf size 1 to 80. Leaf size was first 1 to 40, and 40 was chosen on 3 Titanic splits, so the grid was extended.
Intervals	Day 99's bootstrap over the 30 differences. The splits share rows, so they are too narrow.

ACTUAL OUTPUT

```
Titanic: 891 rows
method              test acc     std  leaves
full tree             0.7695  0.0194   145.5
pruned, best CV       0.8092  0.0233    16.0
pruned, one-SE        0.8045  0.0214     8.1
depth limit           0.8040  0.0233    17.8
leaf-size limit       0.7988  0.0222    48.2
  pruned, best CV    minus full tree: +0.0397 [+0.0310, +0.0489]; better on 28, worse on 2 of 30 splits
  pruned, one-SE     minus full tree: +0.0350 [+0.0260, +0.0438]; better on 28, worse on 2 of 30 splits
  depth limit        minus full tree: +0.0345 [+0.0252, +0.0430]; better on 26, worse on 3 of 30 splits
  leaf-size limit    minus full tree: +0.0292 [+0.0218, +0.0368]; better on 27, worse on 2 of 30 splits
  pruned, best CV minus depth limit: +0.0052 [-0.0004, +0.0108]; better on 19, worse on 6 of 30 splits
  chosen depth limits: [('3', 12), ('4', 9), ('5', 4), ('6', 3), ('8', 2)]
  chosen leaf sizes: [(2, 1), (5, 12), (10, 12), (20, 2), (40, 2), (80, 1)]
Breast cancer: 569 rows
method              test acc     std  leaves
full tree             0.9187  0.0180    17.5
pruned, best CV       0.9263  0.0159     7.7
pruned, one-SE        0.9230  0.0197     4.2
depth limit           0.9205  0.0182    10.2
leaf-size limit       0.9216  0.0210    11.9
  pruned, best CV    minus full tree: +0.0076 [+0.0025, +0.0127]; better on 19, worse on 4 of 30 splits
  pruned, one-SE     minus full tree: +0.0043 [-0.0035, +0.0119]; better on 17, worse on 8 of 30 splits
  depth limit        minus full tree: +0.0018 [-0.0029, +0.0060]; better on 14, worse on 7 of 30 splits
  leaf-size limit    minus full tree: +0.0029 [-0.0035, +0.0092]; better on 12, worse on 10 of 30 splits
  pruned, best CV minus depth limit: +0.0058 [+0.0025, +0.0094]; better on 15, worse on 5 of 30 splits
  chosen depth limits: [('2', 9), ('3', 4), ('4', 9), ('5', 1), ('6', 6), ('8', 1)]
  chosen leaf sizes: [(1, 6), (2, 3), (5, 10), (10, 6), (20, 5)]
the splits share rows, so they are not independent: these intervals are too narrow, a rough guide only
Actual output from this lesson's run.
```



Left: Titanic along the pruning path. Middle: the five methods over 30 splits. Right: label noise, section 6.

What it means

On noisy Titanic every control helps: pruning with the best CV adds +0.0397 and wins 28 of 30 splits, using 16.0 leaves instead of 145.5. On breast cancer the same method adds only +0.0076: there was less to overfit. Against the depth limit pruning is a little better on both (+0.0052 and +0.0058), though the Titanic interval touches zero and the intervals are too narrow anyway. The smaller point matters as much: one-SE gives 8.1 leaves for 0.8045 on Titanic, about half the size for a few tenths of a point.

WHY CHOOSE THIS?

Thirty paired comparisons say far more than one split.

WHY NOT THIS? (TRADEOFFS)

Thirty splits of 891 rows are still 891 rows; a real difference this small needs more data.

6. Label noise: where pruning earns its keep

DEFINITION — start here, no prior knowledge assumed

Label noise means some training answers are wrong. Here a share of training labels is flipped at random; test labels stay correct, so test accuracy shows what was learned about the true pattern.

REAL-WORLD SCENARIO

A clerk who mistypes one form in five. A rule that follows every form learns the typos as well.

IMPLEMENTATION

```
yn = y.copy(); flip = rng.choice(tr, int(round(p * len(tr))), replace=False); yn[flip] = 1 -
    yn[flip]
full = grow(X[tr], yn[tr], 2, gini)
alphas, trees, b, m, se = cv_alpha(X[tr], yn[tr], seed=s)
i = int(np.argmax(m)); ok = np.where(m >= m[i] - se[i])[0]; j = int(ok[np.argmax(b[ok])])
t = at_alpha(alphas, trees, b[j])
```

ACTUAL OUTPUT

```
test labels stay correct; mean of 30 splits; one-SE pruning chosen by 5-fold CV on the noisy
    training part
 flipped  full acc  full leaves  pruned acc  pruned leaves     gain
      0%    0.9187         17.5      0.9230            4.2  +0.0043
     10%    0.8285         47.7      0.9164            3.0  +0.0879
     20%    0.7511         64.1      0.9029            2.1  +0.1519
     30%    0.6524         72.1      0.8916            2.8  +0.2392
     40%    0.5764         76.8      0.7873            2.0  +0.2109
Actual output from this lesson's run.
```

What it means

A full tree must isolate every flipped label, so leaves grow from 17.5 to 76.8 and accuracy falls almost in step with the noise, to 0.5764. Pruning keeps 0.8916 at 30% flipped, a gain of +0.2392. Near 50% the labels carry no signal at all, and pruning also starts to fail at 40% (0.7873). This is the clearest picture of what overfitting costs: the unpruned tree treats errors in the data as facts.

WHY CHOOSE THIS?

Real data has wrong labels, and pruning needs no knowledge of which ones.

WHY NOT THIS? (TRADEOFFS)

Random flips are the easy case; errors that follow a pattern look like signal and survive pruning.

7. The horizon effect: stopping early against pruning

DEFINITION — start here, no prior knowledge assumed

The **horizon effect**: a rule that stops growing can miss a split that looks useless now but makes the next ones excellent. An exclusive-or is the classic case: neither feature helps alone, and both together nearly solve it.

REAL-WORLD SCENARIO

A chess player who only looks one move ahead refuses a sacrifice that wins three moves later.

SPECIFICATION

Data	Synthetic on purpose, because the true structure must be known: y = (x0 > 0.5) xor (x1 > 0.5), 5% noise, four useless features.
Sizes	400 training rows, 2,000 test rows, 30 datasets.
Early stopping	Cut at the first question whose own gain is below the threshold, root first.

ACTUAL OUTPUT

```
synthetic on purpose: y = (x0 > 0.5) xor (x1 > 0.5), 5% noise, four useless features; 400 training rows, 2,000
    test rows, 30 datasets
the first split removes only 0.0095 Gini on average (the largest was 0.0158), though two questions nearly
    solve the problem
the first split used a useful feature (x0 or x1) in 8 of 30 datasets; four of the six features are useless
rule                          test acc   leaves
stop if gain < 0.0              0.7863     56.5
stop if gain < 0.002            0.7869     56.1
stop if gain < 0.005            0.7839     50.8
stop if gain < 0.01             0.5575      5.9
stop if gain < 0.02             0.4974      1.0
stop if gain < 0.05             0.4974      1.0
stop, gain chosen by CV         0.7901     54.0
grow all, prune (best CV)       0.7984     28.3
grow all, prune (one-SE)        0.7565     10.8
thresholds chosen by CV: [(0.0, 16), (0.005, 13), (0.01, 1)]
Actual output from this lesson's run.
```

What it means

The first split removes only 0.0095 on average, so a threshold of 0.02 stops at the root and scores 0.4974: a coin toss. A threshold of 0.01 looks just as harmless and scores 0.5575. Pruning with the best CV keeps the structure (0.7984). But the fair comparison is less dramatic: a threshold *chosen by CV* avoids the trap (0.7901), mostly by choosing almost no threshold, and keeps about twice the leaves (54.0 against 28.3). So the danger is a fixed rule of thumb, not early stopping as such. Two more honest limits: the full tree (0.7863) beat one-SE pruning (0.7565) here, and nothing got near the roughly 0.95 two perfect questions would reach. Greedy growth chose a useful first feature in only 8 of 30 datasets, and pruning can only remove what was grown.

WHY CHOOSE THIS?

Pruning judges a split by what it led to, not only by its own gain.

WHY NOT THIS? (TRADEOFFS)

It cannot add a split that greedy growth never made, and one-SE can over-prune weak signals.

Interview preparation — technical questions

Q: What is the weakest link?

A: The branch with the smallest g(t): errors saved per extra leaf. It is the first to stop paying as alpha rises.

Q: Why not test every pruned tree?

A: Their number multiplies at every question. The weakest-link path needs only one tree per alpha range; here 19 trees for a 145-leaf tree.

Q: How is alpha chosen?

A: K-fold CV at geometric means of neighbouring alphas, each fold using its own path.

Q: What is the one-SE rule?

A: Take the simplest model within one standard error of the best score. Here 4 leaves instead of 15 at about the same accuracy.

Q: Why can a full tree miss 100% training accuracy?

A: Identical rows with different labels. Here 11 unavoidable errors on 623 passengers.

Interview preparation — theoretical questions

Q: Post-pruning or pre-pruning?

A: Post-pruning judges splits by what followed them, and scored slightly higher here. Pre-pruning is cheaper, and cross-validating it avoids most of the danger.

Q: How do you recognise overfitting and underfitting?

A: Overfitting: training high, validation lower, gap growing. Underfitting: both low. The Titanic path shows both.

Q: When does pruning matter most?

A: With noisy labels: a gain of 0.2392 at 30% flipped against 0.0043 on the clean data.

Q: Why does a Random Forest not prune?

A: It removes variance by averaging, not by simplifying each tree: tomorrow's subject.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Pruning turns tree size into one price per leaf. It adds almost nothing where labels are clean and a great deal where they are noisy, and it gives smaller trees a person can read. It cannot invent splits that greedy growth never made, and a depth limit chosen by cross-validation is nearly as good and far simpler.

- Path correct in 1000 of 1000 checks; 19 trees from a 145-leaf tree.
- Titanic: test peaks at 0.8172 (24 leaves), full tree 0.7537, while training reaches 0.9823.
- 30 splits: pruning +0.0397 on Titanic, +0.0076 on breast cancer; against a depth limit, about +0.005.
- 30% flipped labels: full tree 0.6524, pruned 0.8916.
- XOR: a fixed threshold of 0.02 gives 0.4974; a threshold chosen by CV 0.7901; pruning 0.7984.

Suggested watch order, if starting from zero

- 1. How to Prune Regression Trees, Clearly Explained!!! (StatQuest) — the idea on regression trees, step by step.
- 2. Cost Complexity Pruning (Theory + Code) (Meerkat Statistics) — the cost and the code.

Day 107 preview

- **Random forests.** Instead of cutting one tree back, grow many and average them, with a real measurement of the variance that removes.